SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT133

Timed Online Assignment - January Semester 2023

Structured Programming

Thursday, 18 May 2023 **04:00 pm - 06:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **4** questions and comprises **6** pages (including cover page).

2. You must answer **4** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the Word file provided it does not exceed the file size limit of 500MB. **Images of handwritten answers will not be marked**.

6. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

ICT133 Copyright © 2023 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

A right-angled triangle is a triangle in which one angle is a right angle (that is, a 90-degree angle) or two sides are perpendicular. To verify if a triangle is a right-angled triangle, we can use the Pythagorean theorem:

$a^2 + b^2 = c^2$ where a, b and c are the lengths of the triangle.

Question 1a

Write a Python program that reads in 2 values: one shorter (a or b) side and the longest side (c). Your program will then demonstrate how long the other shorter side should be, in order to form a right-angled triangle. Assume all input values are correct.

A sample program execution is as follows:

Enter the longest side: ***5***
Enter one shorter side: ***3***
In order to form a right-angled triangle, the other shorter side should be 4.0

(8 marks)

Question 1b

Write and express a Python program that reads in 3 sides of a triangle. Your program will determine if the 3 sides can form a right-angled triangle. Assume all input values are correct.

Sample program run 1:

Enter side a: ***5***
Enter side b: ***3***
Enter side c: ***4***
These 3 sides [3.0, 4.0, 5.0] can form a right-angled triangle

Sample program run 2:

Enter side a: ***3.3***
Enter side b: ***5.3***
Enter side c: ***4***
These 3 sides [3.3, 4.0, 5.3] CANNOT form a right-angled triangle

(15 marks)

Question 2

Do not use any list, set or dictionary collection for this question. Read both part Q2(a) and Q2(b) before attempting the question. You can submit part Q2(a) and Q2(b) together.

Question 2a

Design and write a Python program that simulates the self-ordering & checkout terminal for a Café.

The Café has the following food item in its menu:

Code	**Item**	**Price**
A	Soup of the day	$3.50
B	Garden Salad	$4.50
C	BLT Sandwich	$5.50

Your program should present the above menu before allowing customer to enter item's code to purchase, until the customer enters "X" or "x" to stop.

The final price will then be computed and displayed.

A sample program execution is as follows:

*<< **Café Menu** >>*
A. Soup of the day
B. Garden Salad
C. BLT Sandwich
X. Exit
Enter your order: ***A***
Enter your order: ***C***
Enter your order: ***B***
Enter your order: ***A***
Enter your order: ***X***
Thank you, please pay $17.00

(14 marks)

Question 2b

- Implement and enhance part Q2(a) to offer 10% discount to Café members if the final price is more than $20.Your program should only ask for membership if the order cost more than $20.
- If the customer is not a member, then the pricing will follow those in part Q2(a).

Sample run 1:	Sample run 2:
<< Café Menu >> *A. Soup of the day* *B. Garden Salad* *C. BLT Sandwich* *X. Exit* *Enter your order:* ***A*** *Enter your order:* ***C*** *Enter your order:* ***B*** *Enter your order:* ***A*** *Enter your order:* ***B*** *Enter your order:* ***X*** *Are you a member? (Y/N):* ***Y*** *Thank you, please pay $19.35*	*<< Café Menu >>* *A. Soup of the day* *B. Garden Salad* *C. BLT Sandwich* *X. Exit* *Enter your order:* ***A*** *Enter your order:* ***C*** *Enter your order:* ***B*** *Enter your order:* ***A*** *Enter your order:* ***B*** *Enter your order:* ***X*** *Are you a member? (Y/N):* ***N*** *Thank you, please pay $21.50*

(8 marks)

Question 3

Read both part Q3(a) and Q3(b) before attempting the question. You can submit part Q3(a) and Q3(b) together.

Question 3a

Write the following function:

getDiceValues(number: int): list of sorted values

This function returns a list, containing the "number" of randomly picked dice values, sorted in <u>*descending*</u> order.

For example,

- *getDiceValues(3)* may returns [5,5,2]
- *getDiceValues(5)* may returns [6,4,4,3,1]

(10 marks)

ICT133 Copyright © 2023 Singapore University of Social Sciences (SUSS)

Question 3b

Applying the *getDiceValues()* function in part Q3(a), create and write a Python program that allows 2 players to solve/play the following game:

(i) At start of game, each player will get 10 dice values (to play10 rounds).
(ii) In each round, the players will compare their highest dice value.
(iii) Player with the bigger dice value wins this round.
(iv) The game ends when a player wins 2 consecutive rounds.
(v) Discard the dice value used in step (ii) & (iii).
(vi) Repeat step ii to v, until all 10 rounds are played.
(vii) If there is no winner, conclude the game as a draw.

Sample program execution:	***Game logic/explanation***
Enter player 1 name: ***Jack*** *Enter player 2 name:* ***Jim*** *Round 1 - Jack 0 : Jim 0* *Round 2 - Jack 0 : Jim 0* *Round 3 - Jack 1 : Jim 0* *Round 4 - Jack 0 : Jim 1* *Round 5 - Jack 0 : Jim 2* *Jim is the winner!!*	*assumes Jack has [6, 6, 5, 3, 3, 2, 2, 2, 1, 1]* *assumes Jim has [6, 6, 4, 4, 4, 4, 4, 4, 2, 2]* *Jack's 6 vs Jim's 6 =>> a draw* *Jack's 6 vs Jim's 6 =>> a draw* *Jack's 5 vs Jim's 4 =>> Jack wins this round* *Jack's 3 vs Jim's 4 =>> Jim wins this round* *Jack's 3 vs Jim's 4 =>> Jim wins this round* *Jim has won 2 consecutive round*

(15 marks)

Question 4

Read both part Q4(a) , Q4(b) and Q4(c) before attempting the question. You can submit part Q4(a), Q4(b) and Q4(c) together.

Question 4a

Create and write a function *countRepeatingChar(word: string): int*, that returns the highest number of the "repeating letter" in the given parameter word. Assume the given parameter word is in lowercase.

For example, *countRepeatingChar(word)* returns these values for the following words as parameters:

- assistants ==> 4
- that ==> 2
- business ==> 3
- count ==> 1

(8 marks)

Question 4b

Write a function *initializeDictionary(filename: string): dictionary,* that reads the words from the given filename and returns the dictionary in the following format:

```
stats = {  4: ['assistants'],
           2: ['that', 'function', 'repeating', 'letters'],
           3: ['repeated', 'suss', 'business', 'assist'],
           1: ['count', 'word'] }
```

The key is the number of the "repeating letters", and the value is a list of words.

Content of the file will be words separated by space. Each line may contain one or more words (all in lowercase). Example of a sample input file content:

```
assistants
repeated
that function count repeating letters
suss business
word assist
```

For each of the word read, find the highest number of the "repeating letter" for this word. Use this number as the key to the dictionary, to either build the list or append the word into the list.

(13 marks)

Question 4c

Create and implement a main function that calls the *initializeDictionary* function, and write selected content of the dictionary into another file, in this order and format:

- ignore words that has no repeating letter, i.e., those words with key = 1
- in ascending order of number of “repeating letters”, follow by the list of words that has this number of repeating letters

Sample output file:

```
2: that function repeating letters
3: repeated suss business assist
4: assistants
```

(9 marks)

----- END OF PAPER -----